

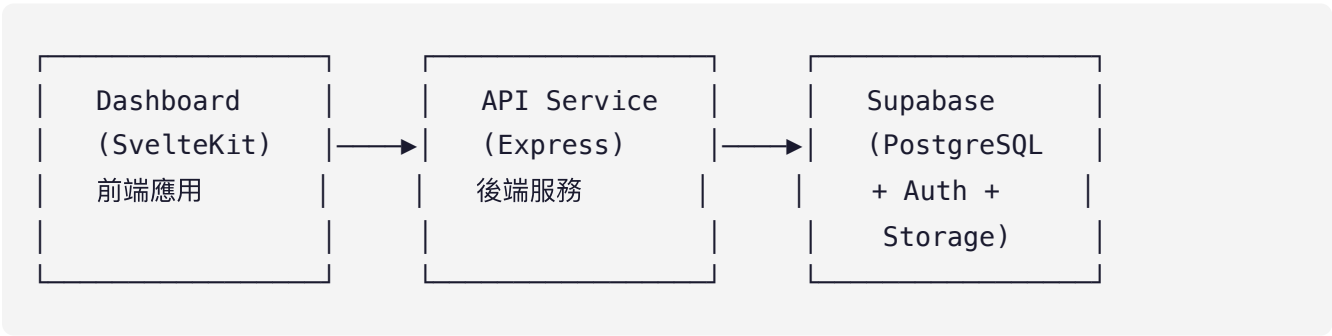
部署指南

目錄

- 1. 部署架構概述
- 2. 環境要求
- 3. Supabase 雲端部署
- 4. Dashboard 部署
- 5. API 服務部署
- 6. 域名與 HTTPS 配置
- 7. Docker 部署
- 8. 部署後驗證

部署架構概述

ClassroomIO 採用前後端分離架構，由三個核心組件組成：



推薦部署方案：

組件推薦平台替代方案

DashboardVercelNetlify / Cloudflare Pages / 自建伺服器

API ServiceRailway / Fly.ioDocker 自建 / VPS

SupabaseSupabase Cloud自建 Supabase (Docker)

環境要求

系統要求

項目最低要求推薦配置

Node.js20.x20.19.3+

pnpm8.x最新版

Docker20.x最新版（自建 Supabase 用）

CPU2 核4 核

記憶體4 GB8 GB

磁碟20 GB50 GB

網路要求

服務埠號說明

Dashboard5173 (dev) / 3000 (prod)HTTP/HTTPS

API3001HTTP

Supabase54321 (API) / 54322 (DB)HTTP / PostgreSQL

Supabase 雲端部署

步驟 1：建立 Supabase 項目

1. 前往 supabase.com 註冊/登入
2. 點擊「New Project」建立新項目
3. 選擇區域（推薦：離目標用戶最近的區域）
4. 設定資料庫密碼（請妥善保管）

步驟 2：取得 API Keys

進入項目設定頁面（Settings → API）：

Project URL: `https://.supabase.co`

anon public key: 用於前端公開存取

service_role key: 用於後端管理操作（請保密）

步驟 3：執行資料庫遷移

安裝 Supabase CLI

```
pnpm add -g supabase
```

登入 Supabase

```
supabase login
```

連結到雲端項目

```
cd classroomio-main  
supabase link --project-ref
```

執行所有遷移

```
supabase db push --include-all
```

步驟 4：驗證資料庫

遷移完成後，在 Supabase Dashboard 的 SQL Editor 中執行：

```
-- 檢查所有表格是否建立完成
```

```
SELECT table_name FROM information_schema.tables  
WHERE table_schema = 'public'  
ORDER BY table_name;
```

應看到以下核心表格（共 30+ 個）：

`profile` – 用戶資料

`organization` – 組織

`organizationmember` – 組織成員

`course` – 課程

`lesson` – 課時

`exercise` – 練習

`question` – 題目

`submission` – 提交

`community_question` / `community_answer` – 社群討論

步驟 5：設定認證

在 Supabase Dashboard → Authentication → Providers 中：

1. 確認 Email 認證已啟用
2. 如需第三方登入（Google、GitHub 等），配置對應的 OAuth 提供者
3. 設定 SMTP 郵件服務（Settings → Auth → SMTP）

Dashboard 部署

方案 A：Vercel 部署（推薦）

1. Fork 或匯入專案

將程式碼推送到 GitHub 倉庫

2. 連結 Vercel

- 前往 vercel.com
- 點擊「Import Project」
- 選擇 GitHub 倉庫

3. 配置構建設定

設定值

Framework PresetSvelteKit

Root Directory `apps/dashboard`

Build Command `pnpm build`

Output Directory `build`

4. 設定環境變數

在 Vercel 項目設定中新增以下環境變數：

```
PUBLIC_SUPABASE_URL=https://.supabase.co
PUBLIC_SUPABASE_ANON_KEY=
PRIVATE_SUPABASE_SERVICE_ROLE=
PUBLIC_IS_SELFHOSTED=true
PRIVATE_APP_HOST=your-domain.com
PRIVATE_APP_SUBDOMAINS=app
```

5. 部署

推送程式碼後 Vercel 會自動部署

方案 B：Node.js 自建部署

```
cd apps/dashboard
```

安裝依賴

```
pnpm install
```

構建

```
pnpm build
```

啟動（預設埠號 3000）

```
PORT=3000 node build
```

使用 PM2 進程序管理：

安裝 PM2

```
pnpm add -g pm2
```

啟動應用

```
pm2 start ecosystem.config.cjs --name classroomio-dashboard
```

設定開機自啟

```
pm2 startup  
pm2 save
```

PM2 配置檔 `ecosystem.config.cjs` :

```
module.exports = {  
  apps: [{  
    name: 'classroomio-dashboard',  
    script: 'build/index.js',  
    cwd: '/path/to/classroomio-main/apps/dashboard',  
    env: {  
      PORT: 3000,  
      NODE_ENV: 'production'  
    },  
    instances: 'max',  
    exec_mode: 'cluster'  
  }]  
};
```

API 服務部署

方案 A : Railway 部署

1. 前往 railway.app
2. 建立新項目 → Deploy from GitHub
3. 選擇倉庫，設定 Root Directory 為 `apps/api`
4. 設定環境變數（見下方）
5. Railway 會自動偵測 Node.js 應用並部署

方案 B : Docker 部署

```
cd apps/api
```

構建映像

```
docker build -t classroomio-api .
```

運行容器

```
docker run -d \  
  --name classroomio-api \  
  -p 3001:3001 \  
  --env-file .env \  
  classroomio-api
```

方案 C : Node.js 自建部署

```
cd apps/api
```

安裝依賴

```
pnpm install
```

構建

```
pnpm build
```

啟動

```
pnpm start
```

API 環境變數

```
PUBLIC_SUPABASE_URL=https://.supabase.co  
PUBLIC_SUPABASE_ANON_KEY=  
PRIVATE_SUPABASE_SERVICE_ROLE=
```

影片上傳（可選）

```
CLOUDFLARE_BUCKET_DOMAIN=  
CLOUDFLARE_ACCESS_KEY=  
CLOUDFLARE_SECRET_ACCESS_KEY=  
CLOUDFLARE_ACCOUNT_ID=
```

郵件（可選）

```
SMTP_HOST=  
SMTP_USER=  
SMTP_PASSWORD=  
SMTP_PORT=
```

域名與 HTTPS 配置

自訂域名設定

1. Dashboard 域名

在 DNS 中新增 CNAME 記錄：

```
app.your-domain.com → cname.vercel-dns.com
```

在 Vercel 項目設定中新增自訂域名

2. API 域名

在 DNS 中新增 A 記錄或 CNAME：

```
api.your-domain.com →
```

HTTPS 配置

Vercel: 自動提供 SSL 證書 (Let's Encrypt)

自建伺服器: 使用 Certbot 或 Cloudflare 代理

使用 Certbot 取得 SSL 證書

```
sudo certbot --nginx -d app.your-domain.com -d api.your-domain.com
```

Supabase CORS 配置

在 Supabase Dashboard → Settings → API 中，設定允許的 CORS 來源：

```
https://app.your-domain.com  
https://api.your-domain.com
```

Docker 部署

使用 Docker Compose

專案根目錄的 `docker/` 目錄包含 Docker 配置。

建立 `docker-compose.yml`：

```
version: '3.8'

services:
  dashboard:
    build:
      context: .
      dockerfile: apps/dashboard/Dockerfile
    ports:
      - "3000:3000"
    environment:
      - PUBLIC_SUPABASE_URL=${PUBLIC_SUPABASE_URL}
      - PUBLIC_SUPABASE_ANON_KEY=${PUBLIC_SUPABASE_ANON_KEY}
      - PRIVATE_SUPABASE_SERVICE_ROLE=${PRIVATE_SUPABASE_SERVICE_ROLE}
    depends_on:
      - api

  api:
    build:
      context: .
      dockerfile: apps/api/Dockerfile
    ports:
      - "3001:3001"
    environment:
      - PUBLIC_SUPABASE_URL=${PUBLIC_SUPABASE_URL}
      - PUBLIC_SUPABASE_ANON_KEY=${PUBLIC_SUPABASE_ANON_KEY}
      - PRIVATE_SUPABASE_SERVICE_ROLE=${PRIVATE_SUPABASE_SERVICE_ROLE}
```

啟動服務：

```
docker-compose up -d
```

部署後驗證

檢查清單

- ☐ Dashboard 可正常訪問
- ☐ API 服務正常回應 (GET `/health` 或根路徑)
- ☐ 用戶註冊功能正常
- ☐ 用戶登入功能正常
- ☐ 課程建立功能正常
- ☐ 課時內容上傳正常
- ☐ 圖片/影片上傳正常
- ☐ 社群功能正常
- ☐ SMTP 郵件發送正常

常見問題

Q: 登入後顯示空白頁面

A: 檢查 `PUBLIC_SUPABASE_URL` 和 `PUBLIC_SUPABASE_ANON_KEY` 是否正確

Q: 圖片無法上傳

A: 確認 Supabase Storage 已啟用，檢查 RLS 策略

Q: 影片無法上傳

A: 確認 API 服務和 Cloudflare R2 配置正確

Q: 郵件無法發送

A: 檢查 SMTP 配置，確認防火牆允許 SMTP 埠號